

Campus Services Portal

A Web-Based Portal for University Students and Administrators

BUSINESS REQUIREMENTS DOCUMENT

Suggested Tech Stack: ASP.NET Core Web API · SQL Server · HTML/CSS/JavaScript · Fetch API

Document Type: Business Requirements Document (BRD)

1. Project Overview

The Campus Services Portal is a full-stack web application built for a Sri Lankan university. It gives students one place to manage their day-to-day university interactions — hostel applications, lab bookings, event sign-ups, complaints, certificate requests and fee payments — while giving administrators the tools to review, approve and track all of that activity from a single dashboard.

As a student project, this brief is intentionally close to a real client engagement: the requirements are grouped into modules, each with its own data, rules and workflow, and you are expected to design the database, build the API, and connect it to a plain HTML/CSS/JavaScript frontend using the Fetch API — mirroring how a junior developer would receive work in industry.

2. Objectives

By the end of this project, students should be able to demonstrate the following outcomes:

Objective	What This Demonstrates
Develop a REST API	Ability to design resource-based URLs and implement GET/POST/PUT/DELETE actions in ASP.NET Core Web API.
Build a responsive frontend	Ability to build usable, mobile-friendly UI using only HTML, CSS and vanilla JavaScript.
Perform CRUD operations	Ability to connect an API to a real SQL Server database using Entity Framework Core.
Apply business rules and validations	Ability to translate written rules into server-side and client-side validation logic.
Integrate frontend and backend using Fetch API	Ability to consume a REST API asynchronously and update the DOM based on JSON responses.

3. User Roles & Permissions

Two roles interact with the system. Keep the permission boundaries below in mind when designing your authentication/authorization logic (e.g. role-based checks using JWT claims).

3.1 Student

- View and update own profile.
- Apply for hostel accommodation, labs, events, complaints and certificates.
- Track the status of every request they have submitted.
- View own fee history and download receipts.
- Receive notifications relevant to their own activity only.

3.2 Administrator

- Manage (create, update, deactivate) student records.
- Approve or reject hostel, certificate and complaint requests.

- Manage master data: hostels, rooms, labs, event listings, complaint categories.
- View system-wide dashboard statistics across all modules.
- Change the status of any request — this is the only role permitted to do so (see Business Rules).

4. Functional Modules

Each module below follows the same structure: a description, the features a student must implement, the business rules that govern it, a starter list of REST endpoints, and the database tables it relies on. Treat the endpoint tables as a minimum — you may add endpoints (e.g. pagination, filtering) as needed. Sections marked **NEW** are new requirements added in this revision; all original module content is preserved unchanged.

Module 1: Student Profile

Handles student registration and self-service profile management. This module is usually built first because every other module depends on a valid Student record.

Key Features

- Register a new student (name, index number, email, faculty, contact details).
- Update own profile information.
- Search/filter students by name, index number or faculty (admin only).
- View a single student's full profile, including a summary of their activity across other modules.

Business Rules

- Index numbers and emails must be unique across all students.
- Students cannot edit their own index number once registered.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/students/register	Register a new student account.
GET	/api/students/{id}	Get a single student's profile.
PUT	/api/students/{id}	Update a student's own profile.
GET	/api/students?search=&faculty=	Search/filter students (admin only).
DELETE	/api/students/{id}	Deactivate a student account (admin only).

Related Database Table(s): Students

Student Registration Validation

New students must not be able to self-register with arbitrary details. Registration must be validated against a preloaded, university-maintained list of legitimate students (e.g. imported from the Registrar's office) before an account is created.

Key Features

- Validate the Student Index Number entered at registration against a preloaded student master database.

- Reject registration immediately with a clear error if the index number is not found in the master list, or if it has already been used to create an account.
- Pre-fill known fields (name, faculty) from the master record where available, reducing manual entry errors.
- Administrator can bulk-import/refresh the student master list (e.g. via CSV upload) ahead of each intake.

Business Rules

- A registration request is rejected if the Student Index Number does not exist in the preloaded StudentMasterList.
- A Student Index Number in the master list can only be linked to one Students account (prevents duplicate registration).
- Only administrators can add, update or bulk-import records in the student master list.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/student-master/{indexNumber}	Verify an index number exists in the master list (used during registration).
POST	/api/student-master/import	Bulk-import/refresh the student master list from a file (admin only).
GET	/api/student-master?search=	Search the master list (admin only).

Related Database Table(s): StudentMasterList (NEW)

Email Verification (Registration Activation Gate)

After a registration request passes Student Registration Validation (the Index Number is found in StudentMasterList), the new account must still be email-verified before it can log in — closing the gap between ‘record created’ and ‘account usable’.

Key Features

- On successful registration, generate a single-use, time-limited email verification token and send it to the student’s registered email address.
- Provide an endpoint the frontend calls when the student opens the verification link, which marks the account as verified.
- Allow a student to request the verification email again if the original token expired or was lost.
- Block login for an unverified account with a clear, distinct error message (not the generic invalid-credentials message).

Business Rules

- A newly registered Students account is created with EmailVerified = false and cannot authenticate until EmailVerified = true.
- A verification token can only be used once and expires after a configurable period (e.g. 24 hours); an expired or already-used token is rejected.
- Requesting a new verification email invalidates any previously issued, unused token for that account.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/auth/verify-email	Verify an account using the token from the emailed link.
POST	/api/auth/resend-verification	Re-send a verification email for an unverified account.

Related Database Table(s): Students (extended with EmailVerified, EmailVerificationToken, EmailVerificationTokenExpiresAt)

Student Activation / Deactivation

Administrators can suspend a student's portal access without deleting their historical data, and safely reactivate them later — while making sure a student is never deactivated mid-way through something they are still actively using.

Key Features

- Administrator deactivates a student account (a soft action — the record and all history are retained).
- Administrator reactivates a previously deactivated account.
- A deactivated student cannot log in; any attempt returns a clear 'account deactivated' message.
- Deactivation is blocked, with a specific reason returned, if the student has any of the disqualifying active items listed below.

Business Rules

- A student cannot be deactivated while they have an active hostel allocation (an Approved or Room Assigned hostel application).
- A student cannot be deactivated while they have upcoming lab bookings (future-dated, status Held or Confirmed).
- A student cannot be deactivated while they have upcoming event registrations (future-dated events).
- A student cannot be deactivated while they have a pending certificate request.
- A student cannot be deactivated while they have unpaid fees, including unpaid library fines.
- Only administrators can deactivate or reactivate a student account.
- Reactivating a student does not automatically restore anything that was cancelled prior to deactivation.

Suggested REST API Endpoints

Method	Endpoint	Description
PUT	/api/admin/students/{id}/deactivate	Deactivate a student (rejected with a reason if blocking conditions exist).
PUT	/api/admin/students/{id}/reactivate	Reactivate a previously deactivated student.
GET	/api/admin/students/{id}/deactivation-check	Preview whether a student can currently be deactivated, and why not if they can't.

Related Database Table(s): Students (extended with IsActive, DeactivatedAt); checks read from HostelApplications, LabBookings, EventRegistrations, CertificateRequests, FeePayments

Authentication: Forgot Password / Reset Password

Adds a self-service password reset flow so a student who forgets their password can regain access without administrator involvement, following the same secure-token pattern as Email Verification above.

Key Features

- Student requests a password reset by submitting their registered email address.
- System issues a single-use, time-limited reset token and emails it as a reset link.
- Student submits the token together with a new password to complete the reset.
- The response to a reset request never reveals whether the email address exists in the system, to prevent account enumeration.

Business Rules

- A password reset token can only be used once and expires after a short, configurable period (e.g. 30–60 minutes).
- Submitting a new password invalidates all other active sessions/tokens for that account, forcing re-login everywhere.
- Requesting a new reset token invalidates any previously issued, unused token for that account.
- The new password must satisfy the same validation rules as registration (e.g. minimum length/complexity).

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/auth/forgot-password	Request a password reset token for a given email address.
POST	/api/auth/reset-password	Submit a reset token and new password to complete the reset.

Related Database Table(s): PasswordResetTokens (NEW), Students (existing — referenced by email)

Module 2: Hostel Accommodation

Lets students apply for hostel accommodation and lets administrators review applications and assign rooms once approved.

Key Features

- Submit a hostel application (preferred hostel, term/semester, special requirements).
- View own application status (Pending / Approved / Rejected / Room Assigned).
- Administrator reviews and approves or rejects applications.
- Administrator assigns a specific room only after approval.

Business Rules

- Hostel applications must be approved by an administrator before a room can be assigned.
- A room cannot be assigned to more than one active student at a time (check room capacity).
- Only administrators can change an application's status.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/hostel-applications	Submit a new hostel application.

Method	Endpoint	Description
GET	/api/hostel-applications/student/{studentId}	View a student's own application(s).
GET	/api/hostel-applications	List all applications, filterable by status (admin only).
PUT	/api/hostel-applications/{id}/status	Approve or reject an application (admin only).
PUT	/api/hostel-applications/{id}/assign-room	Assign a room after approval (admin only).

Related Database Table(s): HostelApplications, Hostels, Rooms

Admin: Hostel & Room Management

Key Features

- Administrator creates, updates and deletes (or deactivates) hostel records.
- Administrator creates, updates and deletes rooms within a hostel, including setting each room's maximum capacity.
- View current occupancy vs. capacity for each room.

Business Rules

- A hostel cannot be deleted while it has rooms with active occupants; deactivate instead.
- A room's capacity cannot be reduced below its current number of assigned occupants.
- Room capacity is enforced at assignment time (see Module 2 business rule on room capacity).

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/hostels	Create a new hostel (admin only).
PUT	/api/hostels/{id}	Update a hostel (admin only).
DELETE	/api/hostels/{id}	Delete/deactivate a hostel (admin only).
POST	/api/hostels/{hostelId}/rooms	Create a room within a hostel, with capacity (admin only).
PUT	/api/rooms/{id}	Update a room, including capacity (admin only).
DELETE	/api/rooms/{id}	Delete/deactivate a room (admin only).
GET	/api/rooms/{id}/occupancy	View current occupancy vs. capacity for a room.

Related Database Table(s): Hostels, Rooms (extends existing tables — no schema change required beyond capacity/status fields)

Module 3: Lab Reservation

Allows students to browse available computer/science labs and reserve a specific time slot, and prevents any two students from booking the same slot.

Key Features

- View a list of labs and their available time slots.
- Reserve a time slot for a chosen lab.
- Cancel an existing reservation.
- View own upcoming and past bookings.

Business Rules

- Lab time slots cannot be double-booked — the API must reject a booking if the slot is already taken.
- A student may only cancel their own reservation, not another student's.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/labs	List all labs.
GET	/api/labs/{id}/slots?date=	List available time slots for a lab on a given date.
POST	/api/lab-bookings	Reserve a time slot (rejects if already booked).
DELETE	/api/lab-bookings/{id}	Cancel a reservation (owner only).
GET	/api/lab-bookings/student/{studentId}	View a student's own bookings.

Related Database Table(s): Labs, LabBookings

Lab Management & Seat-Level Reservation

Adds administrator CRUD for labs and a capacity model that distinguishes two lab types: Computer Labs, where a student reserves a specific numbered seat/workstation, and Science Labs, where a booking simply consumes one unit of overall capacity.

Key Features

- Administrator creates, updates and deletes labs, setting a LabType of 'Computer' or 'Science'.
- Administrator sets and updates a lab's overall capacity (used directly for Science Labs).
- For Computer Labs, administrator defines individual seats/workstations (e.g. Seat 1–30); students choose a specific seat when booking a slot.
- For Science Labs, students simply reserve a slot and the system tracks bookings against the lab's total capacity for that slot (no seat selection UI needed).

Business Rules

- A Computer Lab booking must reference a specific, currently-unbooked seat for that date/time slot.
- A Science Lab booking is rejected once the number of existing bookings for that slot equals the lab's capacity.
- A seat cannot be deleted while it has future active bookings.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/labs	Create a lab, including LabType and capacity (admin only).
PUT	/api/labs/{id}	Update a lab, including capacity (admin only).
DELETE	/api/labs/{id}	Delete/deactivate a lab (admin only).
POST	/api/labs/{id}/seats	Add a seat/workstation to a Computer Lab (admin only).
DELETE	/api/labs/{id}/seats/{seatId}	Remove a seat (admin only).
GET	/api/labs/{id}/seats?date=&slot=	List seat availability for a Computer Lab slot.

Related Database Table(s): Labs (extended with LabType, Capacity), LabSeats (LabSeats is NEW; LabBookings extended with an optional SeatId for Computer Labs)

Module 4: Event Registration

Displays upcoming university events and manages student sign-ups, ensuring no student registers twice for the same event.

Key Features

- View a list of upcoming/past events with details (date, venue, capacity).
- Register for an event.
- Cancel an event registration.
- View own list of registered events.

Business Rules

- One student cannot register twice for the same event.
- Registration should be blocked once an event reaches its capacity (recommended extension of the base rule).

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/events	List all events.
POST	/api/event-registrations	Register for an event (rejects duplicate registration).
DELETE	/api/event-registrations/{id}	Cancel a registration (owner only).
GET	/api/event-registrations/student/{studentId}	View a student's registered events.

Related Database Table(s): Events, EventRegistrations

Event & Venue Management

Administrator CRUD for events, a new Venues concept covering both event halls and open spaces, double-booking protection for venues, and enforcement of the existing capacity rule as a hard, mandatory check.

Key Features

- Administrator creates, updates and deletes events (title, date/time, capacity, description).
- Administrator creates, updates and deletes Event Venues, where a venue is either an Event Hall or an Open Space.
- An event is scheduled against a specific venue and date/time range.
- Registration is automatically closed on the frontend and rejected by the API once an event reaches its maximum capacity.

Business Rules

- A venue (Event Hall or Open Space) cannot be booked for a date/time range that overlaps with an existing booking for that same venue.
- Event registration is rejected once RegisteredCount equals the event's Capacity — this makes the previously 'recommended' capacity rule mandatory and enforced server-side.
- Only administrators can create, update, or delete events and venues.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/events	Create an event (admin only).
PUT	/api/events/{id}	Update an event (admin only).
DELETE	/api/events/{id}	Delete/cancel an event (admin only).
POST	/api/venues	Create an Event Hall or Open Space venue (admin only).
PUT	/api/venues/{id}	Update a venue (admin only).
DELETE	/api/venues/{id}	Delete/deactivate a venue (admin only).
GET	/api/venues/{id}/availability?from=&to=	Check a venue for overlapping bookings before scheduling an event.

Related Database Table(s): Venues, Events (extended with Venueld, Capacity) (Venues is NEW)

Event Capacity vs. Venue Capacity Validation

Closes a gap in the Admin: Event & Venue Management section above — nothing currently stops an administrator from setting an event's capacity higher than the physical space it is assigned to. This adds that check.

Key Features

- When creating or updating an event, the API checks the event's Capacity against the Capacity of the selected Venue before saving.
- The event form on the frontend should surface the selected venue's capacity next to the capacity field to guide the administrator, but the authoritative check happens server-side.

Business Rules

- An event's Capacity must never exceed the Capacity of its assigned Venue.
- The API rejects (400 Bad Request) any create or update where Capacity is greater than Venue.Capacity, with a message stating the venue's maximum.

- Changing an event's Venue to a smaller one is rejected if the event's current Capacity would exceed the new venue's Capacity.

Suggested REST API Endpoints

Method	Endpoint	Description
—	(validation only)	Added to the existing POST /api/events and PUT /api/events/{id} from Admin: Event & Venue Management — no new endpoints required.

Related Database Table(s): Events, Venues (existing — no schema change; validation only)

Event Seat Count Validation (Reserved Seating)

For events that use individually numbered seating — mirroring the Computer Lab seat model in Module 3 — this ensures the configured seat list always matches the event's stated capacity, and that capacity can never be dropped below seats already booked.

Key Features

- An event can optionally be marked UsesReservedSeating; when true, the administrator configures individual EventSeats (seat numbers) for that event.
- The API validates that the total number of configured seats for an event exactly equals its Capacity before the event can be published/opened for registration.
- Attempting to reduce an event's Capacity below the number of currently booked seats is rejected.

Business Rules

- For a reserved-seating event, Capacity must equal the count of rows in EventSeats for that event — a mismatch blocks publishing.
- Event Capacity cannot be reduced below the number of seats already assigned to confirmed EventRegistrations.
- Deleting an individual seat is rejected if that seat currently has a confirmed registration.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/events/{id}/seats	Add a seat to a reserved-seating event (admin only).
DELETE	/api/events/{id}/seats/{seatId}	Remove a seat; rejected if booked (admin only).
GET	/api/events/{id}/seats	List seat configuration and availability for a reserved-seating event.

Related Database Table(s): EventSeats (NEW), Events (extended with UsesReservedSeating), EventRegistrations (extended with optional SeatId)

Booking Expiry / Reservation Hold Timeout (Labs & Events)

Applies to both Module 3 (Lab Reservation) and Module 4 (Event Registration). When a student starts reserving a lab slot/seat or an event seat but does not finish the booking or payment step, the reservation is held temporarily rather than confirmed immediately or lost — and is automatically released if the student abandons it.

Key Features

- Creating a lab booking or event registration first places it in a Held status with an ExpiresAt timestamp set to now plus a configurable hold period (default 15 minutes).
- Completing the booking/payment step within the hold window transitions the record to Confirmed.
- A background/scheduled process periodically scans for Held records past their ExpiresAt time and transitions them to Expired, releasing the seat/slot for other students.
- The hold period is configurable system-wide, not hard-coded, via a settings table.

Business Rules

- A Held reservation counts toward slot/seat/capacity availability for as long as it is Held, so two students cannot both hold the same seat.
- An Expired reservation is treated exactly like a cancellation — it does not count against capacity and does not block a new booking of the same slot/seat.
- The hold period is a single configurable system setting shared by both Lab Reservation and Event Registration.
- The expiry job must run frequently enough that an expired hold is released within roughly one minute of expiring (e.g. every 60 seconds).

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/lab-bookings	(existing, extended) creates a Held reservation with an ExpiresAt time rather than an immediately Confirmed one.
POST	/api/event-registrations	(existing, extended) creates a Held reservation with an ExpiresAt time rather than an immediately Confirmed one.
PUT	/api/lab-bookings/{id}/confirm	Confirm a held lab booking before it expires.
PUT	/api/event-registrations/{id}/confirm	Confirm a held event registration before it expires.
GET	/api/admin/system-settings/reservation-hold-minutes	View/update the configurable hold period (admin only).

Related Database Table(s): LabBookings (extended with Status, ExpiresAt), EventRegistrations (extended with Status, ExpiresAt), SystemSettings (NEW)

Background process: a recurring background job (e.g. an ASP.NET Core IHostedService/BackgroundService running every 60 seconds, or a SQL Server Agent job) selects all Held LabBookings/EventRegistrations where ExpiresAt < now, sets their status to Expired, and frees the associated seat/slot/capacity count.

Module 5: Complaint Management

Gives students a way to raise complaints (e.g. maintenance, hostel, academic) and gives administrators a workflow to triage and resolve them.

Key Features

- Submit a complaint with a category and description.
- Track a complaint's status: Pending → In Progress → Resolved.
- Administrator updates status and can add a resolution note.
- View own complaint history.

Business Rules

- Only administrators can change a complaint's status.
- A complaint must belong to a valid, pre-defined category.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/complaint-categories	List available complaint categories.
POST	/api/complaints	Submit a new complaint.
GET	/api/complaints/student/{studentId}	View a student's own complaints.
GET	/api/complaints?status=	List complaints filtered by status (admin only).
PUT	/api/complaints/{id}/status	Update status / add resolution note (admin only).

Related Database Table(s): ComplaintCategories, Complaints

Complaint Category Management

The original brief lists complaint categories as pre-defined master data but does not provide a way to maintain them. This revision adds the administrator CRUD.

Key Features

- Administrator creates, updates and deletes complaint categories.
- Deactivated categories remain visible on historical complaints but are no longer offered when submitting a new complaint.

Business Rules

- A complaint category cannot be hard-deleted while existing complaints reference it — deactivate instead.
- Category names must be unique.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/complaint-categories	Create a complaint category (admin only).
PUT	/api/complaint-categories/{id}	Update a complaint category (admin only).
DELETE	/api/complaint-categories/{id}	Delete/deactivate a complaint category (admin only).

Related Database Table(s): ComplaintCategories (extended with IsActive flag)

Module 6: Certificate Requests

Allows students to request official documents — Bonafide Certificate, Transcript or Completion Letter — and track approval progress.

Key Features

- Request a certificate by type (Bonafide / Transcript / Completion Letter), with a reason.
- Track approval status (Pending / Approved / Rejected / Ready for Collection).
- Administrator reviews and approves or rejects requests.

Business Rules

- Only administrators can approve, reject, or mark a certificate as ready.
- A student should not be able to submit a duplicate pending request of the same certificate type (recommended extension).

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/certificate-requests	Submit a new certificate request.
GET	/api/certificate-requests/student/{studentId}	View a student's own requests.
GET	/api/certificate-requests?status=	List all requests, filterable by status (admin only).
PUT	/api/certificate-requests/{id}/status	Approve/reject/update a request (admin only).

Related Database Table(s): CertificateRequests

Note: Administrator CRUD for the list of valid Certificate Types is covered under the new Master Data Management module (Module 9) below, alongside Faculties.

Module 7: Fee Payment (Simulation)

Simulates a fee payment process — no real payment gateway is required. Students can view outstanding fees, mark them as paid, and download a receipt.

Key Features

- View current outstanding fees.
- Mark a fee as paid (simulation — no real transaction).
- View payment history.
- Download/generate a simple receipt (PDF or on-screen printable view is acceptable).

Business Rules

- A fee cannot be marked as paid more than once (prevent duplicate payment records for the same fee item).

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/fee-payments/student/{studentId}	View a student's fees and payment history.
POST	/api/fee-payments/{id}/pay	Mark a fee as paid (simulation).
GET	/api/fee-payments/{id}/receipt	Generate/download a receipt.

Related Database Table(s): FeePayments

Fee Management

The administrator workflow that actually creates and assigns fees, and generalises the fee model to cover recurring university charges.

Key Features

- Administrator creates fee types (Tuition Fee, Semester Fee, Exam Fee, and other custom fee types).
- Administrator assigns a fee to a single student, or bulk-assigns a fee to a group (e.g. an entire faculty or intake).
- Administrator can view assignment status across all students for a given fee run (paid / outstanding).
- Administrator can edit or cancel an unpaid fee assignment (e.g. correcting an amount before it is paid).

Business Rules

- Only administrators can create fee types or assign fees to students.
- A fee assignment that has already been paid cannot be edited or cancelled — only unpaid assignments can be changed.
- Bulk assignment must not create duplicate fee items for a student who already has an identical, unpaid fee of the same type and billing period.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/fee-types	Create a fee type, e.g. Tuition, Semester, Exam (admin only).
GET	/api/fee-types	List available fee types.
POST	/api/fee-payments/assign	Assign a fee to one student or bulk-assign to a group (admin only).
PUT	/api/fee-payments/{id}	Edit an unpaid fee assignment (admin only).
DELETE	/api/fee-payments/{id}	Cancel an unpaid fee assignment (admin only).
GET	/api/fee-payments?status=&feeTypeId=	List fee assignments across all students, filterable (admin only).

Related Database Table(s): FeeTypes, FeePayments (extended with FeeTypeId) (FeeTypes is NEW)

Lab Fine Management

Adds support for library fines as a distinct, system-generated fee flow (e.g. for any damage), reusing the same payment simulation as other fees.

Key Features

- System (administrator) generates a lab fine for a student, with a reason and amount.
- Student views and pays outstanding lab fines alongside other fees.
- Administrator can waive or adjust a fine before it is paid.

Business Rules

- A lab fine is created with FeeType = 'lab Fine' and follows the same one-time-payment rule as other fees.
- Only administrators can waive or adjust a fine amount, and only before it has been paid.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/lab-fines	Generate a library fine for a student (admin only).
PUT	/api/lab-fines/{id}/waive	Waive or adjust an unpaid fine (admin only).
GET	/api/lab-fines/student/{studentId}	View a student's library fines.

Related Database Table(s): FeePayments (FeeType = lab Fine) (reuses FeePayments/FeeTypes — no separate table required)

Module 8: Notifications

Centralises system-generated messages so students always know when something relevant to them has changed — hostel approval, event registration confirmation, complaint updates, certificate approval, and fee reminders.

Key Features

- Automatically create a notification whenever a relevant status changes in another module.
- List a student's notifications, newest first.
- Mark a notification as read.

Business Rules

- A student should only ever see their own notifications.
- Notifications should be triggered from the relevant module's status-change endpoint (e.g. hostel approval), not created manually by the student.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/notifications/student/{studentId}	List a student's notifications.
PUT	/api/notifications/{id}/read	Mark a notification as read.

Related Database Table(s): Notifications

Backend Notification Creation Service

The notifications as being 'triggered from the relevant module'.

Key Features

- An internal NotificationService (called directly from other modules' services — e.g. HostelService, EventService, ComplaintService, CertificateService, FeePaymentService — not exposed to the frontend) creates a Notifications row whenever a status changes.
- A protected, server-to-server POST endpoint is also provided as an alternative integration point for modules implemented as separate services.
- Each notification created this way carries a Type (e.g. HostelApproved, EventRegistrationConfirmed, ComplaintUpdated) so the frontend can render an appropriate icon/message.

Business Rules

- The notification-creation endpoint/service is never called directly by a student — only by server-side module logic following a status change.
- Every status-changing endpoint identified elsewhere in this document (hostel approval, event registration, complaint status update, certificate status update, fee assignment/payment) must invoke notification creation as part of the same transaction.

Suggested REST API Endpoints

Method	Endpoint	Description
POST	/api/internal/notifications	Server-to-server: create a notification for a student (called by other modules, not by the frontend).

Related Database Table(s): Notifications (extended with a Type field)

Module 9: Master Data Management

Key Features

- Administrator creates, updates and deletes Faculties (used by Student Profile and reporting/filtering).
- Administrator creates, updates and deletes Certificate Types (used by Certificate Requests, replacing the fixed Bonafide/Transcript/Completion Letter list with a maintainable one).

Business Rules

- A Faculty cannot be deleted while students are linked to it — deactivate instead.
- A Certificate Type cannot be deleted while certificate requests reference it — deactivate instead.
- Only administrators can manage Faculties and Certificate Types.

Suggested REST API Endpoints

Method	Endpoint	Description
GET	/api/faculties	List all faculties.
POST	/api/faculties	Create a faculty (admin only).
PUT	/api/faculties/{id}	Update a faculty (admin only).
DELETE	/api/faculties/{id}	Delete/deactivate a faculty (admin only).
GET	/api/certificate-types	List all certificate types.

Method	Endpoint	Description
POST	/api/certificate-types	Create a certificate type (admin only).
PUT	/api/certificate-types/{id}	Update a certificate type (admin only).
DELETE	/api/certificate-types/{id}	Delete/deactivate a certificate type (admin only).

Related Database Table(s): Faculties, CertificateTypes (both NEW; Students.FacultyId and CertificateRequests.CertificateTypeId now reference these tables instead of free text)

5. Dashboards

5.1 Student Dashboard

- Current hostel application status.
- Upcoming lab bookings.
- Registered events.
- Certificate request statuses.
- Unread notifications.

5.2 Administrator Dashboard

- Total registered students.
- Pending hostel applications.
- Open complaints by category/status.
- Upcoming events and registration counts.
- Pending certificate requests.
- Summary fee collection statistics.

6. Consolidated Business Rules

These rules must be enforced primarily in the API layer (never trust client-side validation alone):

#	Rule	Where to Enforce It
1	One student cannot register twice for the same event.	POST /api/event-registrations — check for an existing record before inserting.
2	Lab time slots cannot be double-booked.	POST /api/lab-bookings — check slot availability inside a transaction before inserting.
3	Hostel applications must be approved by an administrator before room assignment.	PUT /api/hostel-applications/{id}/assign-room — reject if status is not 'Approved'.
4	Only administrators can change request statuses.	Apply role-based authorization ([Authorize(Roles="Admin")]) on every status-changing endpoint.

#	Rule	Where to Enforce It
5	A student can only register if their index number exists in the preloaded StudentMasterList.	POST /api/students/register — verify against StudentMasterList before creating the account.
6	Event Halls and Open Spaces cannot be double-booked for overlapping date/time ranges.	POST/PUT /api/events — check venue availability inside a transaction before scheduling.
7	Event registration is blocked once an event reaches maximum capacity.	POST /api/event-registrations — reject if RegisteredCount >= Capacity.
8	A Computer Lab booking must reference a specific, unbooked seat; a Science Lab booking is rejected once capacity is reached.	POST /api/lab-bookings — branch validation logic on Labs.LabType.
9	A fee assignment that has already been paid cannot be edited or cancelled.	PUT/DELETE /api/fee-payments/{id} — reject if Status = 'Paid'.
10	Every status-changing endpoint must trigger a notification for the affected student.	Call the internal NotificationService (or POST /api/internal/notifications) within the same transaction as the status change.
11	A student cannot be deactivated while they have active hostel allocations, upcoming lab/event bookings, a pending certificate request, or unpaid fees.	PUT /api/admin/students/{id}/deactivate — run the deactivation-check across HostelApplications, LabBookings, EventRegistrations, CertificateRequests and FeePayments before deactivating.
12	A Held lab booking or event registration that is not confirmed within the hold period must be automatically expired and its seat/slot released.	Background job (e.g. IHostedService, every 60 seconds) — set Status = Expired for records where ExpiresAt < now.
13	An event's Capacity must never exceed its assigned Venue's Capacity.	POST/PUT /api/events — reject if Capacity > Venue.Capacity.
14	For reserved-seating events, the number of configured EventSeats must exactly equal the event's Capacity, and Capacity cannot drop below already-booked seats.	POST/PUT /api/events/{id}/seats and PUT /api/events/{id} — validate seat count against Capacity before saving.
15	A password reset token can only be used once and must be rejected once expired.	POST /api/auth/reset-password — check token validity, usage and expiry before allowing the new password.
16	A newly registered account cannot log in until its email address has been verified.	Login endpoint — reject authentication with a distinct error if Students.EmailVerified = false.

7. REST API & Documentation Requirements

Build full CRUD APIs for Students, Hostel Applications, Labs, Events, Complaints, Certificates, Fee Payments and Notifications, following standard REST conventions:

- Use plural, resource-based URLs (e.g. /api/students, not /api/getStudents).
- Return proper HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found.
- Use DTOs to control exactly what data is sent to the frontend (never expose password hashes or internal-only fields).

- Document every endpoint using Swagger/OpenAPI so the frontend team (or you) can test each one before wiring up the UI.
- Consider JWT-based authentication so the API knows whether a request is from a Student or an Administrator.

8. Frontend Requirements

The frontend must be built using HTML, CSS and vanilla JavaScript only (no frameworks such as React, Angular or Vue), and must talk to the backend exclusively through the Fetch API.

- Provide search and filtering wherever a list of records is shown (e.g. students, complaints).
- Use responsive layout techniques (CSS Grid/Flexbox and media queries) so the portal works on both desktop and mobile.
- Validate forms on the client side for immediate feedback, in addition to server-side validation.
- Show clear loading and error states for every Fetch call (e.g. a spinner while waiting, a message if the request fails).
- Keep JavaScript organised — one file per module is a reasonable starting structure (students.js, hostel.js, labs.js, etc.).

9. Database Design

The tables below are a suggested starting schema. You are encouraged to refine column names/types and add audit fields (CreatedAt, UpdatedAt) as you design your own ER diagram.

Table	Purpose	Key Relationships
Students	One row per student account.	Referenced by almost every other table via StudentId (FK).
HostelApplications	One row per hostel application submitted.	FK → Students, FK → Hostels.
Hostels	Master list of hostel buildings.	Referenced by Rooms and HostelApplications.
Rooms	Individual rooms within a hostel, with capacity.	FK → Hostels; referenced by HostelApplications once assigned.
Labs	Master list of labs available for reservation.	Referenced by LabBookings.
LabBookings	One row per reserved time slot.	FK → Labs, FK → Students.
Events	Master list of university events.	Referenced by EventRegistrations.
EventRegistrations	One row per student registered for an event.	FK → Events, FK → Students.
ComplaintCategories	Master list of complaint categories.	Referenced by Complaints.
Complaints	One row per complaint submitted.	FK → Students, FK → ComplaintCategories.
CertificateRequests	One row per certificate request.	FK → Students.
FeePayments	One row per fee item / payment.	FK → Students.

Table	Purpose	Key Relationships
Notifications	One row per notification generated for a student.	FK → Students.
StudentMasterList	Preloaded list of legitimate students used to validate registration.	Referenced by Students during registration; not otherwise linked.
LabSeats	Individual seats/workstations within a Computer Lab.	FK → Labs; referenced by LabBookings.SeatId (optional).
Venues	Master list of Event Halls and Open Spaces.	Referenced by Events.Venueld.
FeeTypes	Master list of fee types (Tuition, Semester, Exam, lab Fine, etc.).	Referenced by FeePayments.FeeTypeId.
Faculties	Master list of university faculties.	Referenced by Students.FacultyId.
CertificateTypes	Master list of certificate types.	Referenced by CertificateRequests.CertificateTypeId.
PasswordResetTokens (NEW)	One row per issued password-reset token.	FK → Students.
EventSeats (NEW)	Individual bookable seats for a reserved-seating event.	FK → Events; referenced by EventRegistrations.SeatId (optional).
SystemSettings (NEW)	Key/value store for configurable system settings, e.g. reservation hold period.	Not linked by FK; read by the booking-hold background job.

10. Non-Functional Requirements

These were not explicit in the original brief but are expected of any production-style student project:

Category	Expectation
Security	Passwords hashed (never stored in plain text); role-based authorization on admin-only endpoints.
Performance	List endpoints should support basic paging so large tables don't slow down the UI.
Usability	Consistent navigation and clear feedback messages across all modules.
Maintainability	Use a layered architecture (Controllers → Services → Repositories) rather than putting logic directly in controllers.
Portability	The SQL script and README must allow another developer to set up the project from scratch.

11. Suggested Build Order (For a Solo or Small-Team Project)

- Design the full database schema and relationships before writing any C# code — including the new master data tables (Faculties, CertificateTypes, FeeTypes, Venues, StudentMasterList).

- Build the Student Profile module end-to-end (DB → API → UI) first — every other module depends on it. Include the registration validation against StudentMasterList from the start.
- Implement authentication (registration/login, JWT issuing) before building admin-only features.
- Build the remaining modules one at a time, always finishing API + Swagger test + frontend together before moving on — pairing each module with its admin CRUD sub-module (e.g. Hostels + Hostel/Room management together).
- Add Notifications last, since it depends on status-change events from other modules already existing — wire the internal NotificationService into each status-changing endpoint as that endpoint is built.
- Build the dashboards last — they only aggregate data that already exists in the other tables.
- Finish with README, SQL script export and a final walkthrough test of every business rule, including the new rules in Section 6.

12. Deliverables

Deliverable	Details
Source Code	Full ASP.NET Core Web API solution and frontend (HTML/CSS/JS), in a Git repository.
SQL Script	A script that creates the full schema and seeds minimal sample data (e.g. hostels, labs, complaint categories, faculties, certificate types, fee types).
REST API	All CRUD endpoints described in Section 4, secured and returning correct status codes.
Frontend	Responsive HTML/CSS/JS pages for every module, wired to the API via Fetch.
Swagger Documentation	Auto-generated OpenAPI docs covering every endpoint, with example requests/responses.
README	Setup instructions, tech stack, ER diagram, and a short description of each module.